

PROGRAM-1

AIM – Create the list a few Basic Linux Commands used in Linux.

Software used – Fedora (Linux)

Theory – Linux commands are powerful tools that allow users to interact with the operating system efficiently through the command line interface. These commands enable file and directory management, such as creating directories (mkdir), creating empty files (touch), listing files (ls), removing files (rm) and directories (rmdir), copying (cp), and moving or renaming files (mv). Commands like cat, more, and less are used for viewing file contents with various levels of control. The pwd command shows the current directory, and cd changes the working directory. Commands like passwd help manage user passwords. Additionally, Linux supports the compilation and execution of programs through commands like gcc for compiling C source files into executables followed by running the output binary directly. Together, these commands form the foundation of navigating, managing, and programming in Linux environments effectively.

Source code –

- **Command-** ls (contents of a directory)

Output:-

```
lab1137-22@fedora:~$ ls /home/lab1137-22/Downloads
'CM CheatSheet Updated.pdf'      com.visualstudio.code.flatpakref
'MariaDB Installation.odt'
'CM Notes(unit 1&2).pdf'         'linux(1).txt'
'CM UNIT - 1 - Anuj Sinha.pdf'  linux.txt
lab1137-22@fedora:~$
```

- **Command-** pwd (print working directory)

Output:-

```
lab1137-22@fedora:~$ pwd /home/lab1137-22/Downloads
/home/lab1137-22
lab1137-22@fedora:~$
```

- **Command** - cd (change directory)

Output:-

```
lab1137-22@fedora:~$ cd /home/lab1137-22/Downloads
```

- **Command-** touch (Creating empty files)

Output:-

```
lab1137-22@fedora:~/Downloads$ touch filekp.txt
```

- **Command-** mkdir (Make directory)

Output:-

```
lab1137-22@fedora:~/Downloads$ mkdir dir kunal
```

- **Command-** rm and rmdir (Remove files and directories)

Output:-

```
lab1137-22@fedora:~/Downloads$ rm filekp.txt
lab1137-22@fedora:~/Downloads$ rmdir dir
```

- **Command-** cp (Copy files & directories)

Output:-

```
lab1137-22@fedora:~/Downloads$ cp linux.txt kunal
```

- **Command-** mv (Moving/Renaming files and directories)

Output:-

```
lab1137-22@fedora:~/Downloads$ mv linux.txt lin.txt
```

- **Command-** cat (to create new files and add content to them)

Output :-

```
sara@pnapp:~$ cat >test1.txt
This is test file #1.
sara@pnapp:~$
```

- **Command-** more (text pager used to display the contents of a file one screenful at a time)

Output:-

```
lab1137-22@fedora:~/Downloads$ more lin.txt
create list of few basic command used in Linux

ls list file in directory
pwd print the current working directory
cd change directory
touchfile.txt creates a new file
mkdir dir name
rm File.txt delete the file
rmdir directory delete empty directory
cp src destination copy file or directory
mv old new rename file or
cat file.txt file content
More/less view file content page by page
passwd
gcc File.c -o output = to compile c program
```

- **Command-** less (Terminal pager program used to view the contents of text files or the output of other Commands, one screen at a time. It is an improvement over the older more Command, primarily because it allows for both forward and backward navigation within the file.)

Output:-

```
lab1137-22@fedora:~/Downloads$ less lin.txt
[1]+      Stopped                  less lin.txt
```

- **Command-** passwd (change user account passwords and manage password-related settings.)

Output:-

```
lab1137-22@fedora:~/Downloads$ passwd
Current password:
New password:
```

- **Command-** gcc (GNU Compiler Collection, -o<output_filename>)

Output:-

```
lab1137-22@fedora:~/Downloads$ gcc linux.c -o output
lab1137-22@fedora:~/Downloads$ ./output
Hello, World!
```

PROGRAM - 2

AIM – Write a Script to find the greatest of three numbers (numbers passed as command line parameter)

Software used – Fedora (Linux)

Theory – Shell scripting in Linux provides an efficient way to automate tasks by writing a sequence of commands in a file. One of the important applications of shell scripting is decision-making using conditional statements like if, else if, and else. In this experiment, we aim to determine the greatest of three numbers using shell scripting. Command line arguments or user input are taken as numbers, and logical operators such as -gt (greater than) are used to compare values. Arithmetic operations are performed using the expr command. The script demonstrates the use of variables, user input, arithmetic evaluation, and conditional constructs to solve problems in a structured way.

Source code –

```
echo "Enter first number: "  
read num1  
  
echo "Enter second number:"  
read num2  
  
echo "Enter third number: "  
read num3  
  
if [ $num1 -ge $num2 ] && [ $num1 -ge $num3 ]  
then  
    echo "The greatest number is: $num1"  
elif [ $num2 -ge $num1 ] && [ $num2 -ge $num3 ]  
then  
    echo "The greatest number is: $num2"  
else  
    echo "The greatest number is: $num3"  
fi
```

Output: -

```
[mait@fedora ~]$ vi greater.sh  
[mait@fedora ~]$ ./greater.sh  
Enter first number: 2  
Enter second number: 5  
Enter third number: 7  
The greatest number is: 7  
  
[mait@fedora ~]$
```

PROGRAM - 3(i)

AIM – Write a Script to check whether the given no. is even or odd

Software used – Fedora (Linux)

Theory – Shell scripting in Linux is used to automate repetitive tasks and perform decision-making using conditional statements. By reading user input and applying modulo division, we can determine if a number is divisible by 2. If divisible, it is even; otherwise, it is odd. Shell scripting is a powerful tool in Linux for automating tasks and performing logical operations. Using if-else conditions, arithmetic calculations like modulo can be applied to user input. By checking divisibility with 2, the script effectively determines whether a given number is even or odd.

Source code –

```
echo "Enter a number: "  
read number  
  
if [ $(( $number % 2 )) -eq 0 ]  
then  
    echo "$number is Even"  
else  
    echo "$number is Odd"  
fi
```

Output: -

```
[mait@fedora ~]$ vi evenodd.sh  
[mait@fedora ~]$ chmod +x evenodd.sh  
[mait@fedora ~]$ ./evenodd.sh  
Enter a number: 8  
8 is Even  
  
[mait@fedora ~]$ ./evenodd.sh  
Enter a number: 13  
13 is Odd  
  
[mait@fedora ~]$ |
```

PROGRAM - 3(ii)

AIM – Write a Script to check whether the given no. is prime or not

Software used – Fedora (Linux)

Theory – In Linux, shell scripting is widely used for automation and mathematical problem-solving. By combining loops, arithmetic expressions, and conditional statements, a script can check if a number is prime. The logic involves dividing the input number by integers up to its square root. If divisible, it is not prime; otherwise, it is prime. This demonstrates effective use of control structures in Bash.

Source code –

```
echo "enter the value of a"
read num

is_prime() {
    if [ $num -le 1 ]; then
        return 1 # not prime
    fi
    for ((i=2; i*i<=num; i++))
    do
        if [ $((num % i)) -eq 0 ]; then
            return 1 # not prime
        fi
    done
    return 0 # prime
}
if is_prime $num
then
    echo "$num is a prime number"
else
    echo "$num is not a prime number"
fi
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# vi prime.sh
root@fedora:/home/lab1137-33/Desktop# ./prime.sh
enter the value of a
7
7 is a prime number

root@fedora:/home/lab1137-33/Desktop# ./prime.sh
enter the value of a
12
12 is not a prime number

root@fedora:/home/lab1137-33/Desktop#
```

PROGRAM - 3(iii)

AIM – Write a Script to check whether the given input is a number or a string

Software used – Fedora (Linux)

Theory – Shell scripting in Linux provides powerful features for validating and processing user input. One such feature is pattern matching with regular expressions, which allows classification of data. In this script, conditional expressions with `[[ ]]` and the `=~` operator are used to check if the given input matches the numeric pattern `^[0-9]+$`. If the input consists entirely of digits, it is identified as a number. Otherwise, it is treated as a string. This approach demonstrates the use of regular expressions and conditional statements to differentiate between numeric and non-numeric inputs, making scripts more robust and user-friendly.

Source code –

```
echo "Enter input: "  
read input  
  
if [[ $input =~ ^[0-9]+$ ]]  
then  
    echo "Input is a number"  
else  
    echo "Input is a string"  
fi
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# vi kp.sh  
root@fedora:/home/lab1137-33/Desktop# ./kp.sh  
Enter input:  
123  
Input is a number  
  
root@fedora:/home/lab1137-33/Desktop# ./kp.sh  
Enter input:  
hello  
Input is a string
```

PROGRAM - 3(iv)

AIM – Write a Script to compute no. of character and words in each line of a given file

Software used – Fedora (Linux)

Theory – In Linux, shell scripting simplifies file handling and text processing tasks. This experiment uses the `wc` command, which stands for word count, to compute file statistics. The option `wc -w` gives the number of words, while `wc -c` provides the character count. The script first checks whether the given file exists, then processes its contents and displays the results. Such scripts demonstrate automation in analyzing text files, reducing manual effort and ensuring accurate computation.

Source code –

```
echo "Enter filename: "
read file

if [ ! -f "$file" ]; then
    echo "File not found!"
    exit 1
fi

word_count=$(wc -w < "$file")
char_count=$(wc -c < "$file")

echo "Number of words: $word_count"
echo "Number of characters: $char_count"
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# ./kp.sh
Enter filename:
sample.txt
Number of words: 5
Number of characters: 28
```

```
Hello world
This is Fedora
```


PROGRAM - 3(v)

AIM – Write a Script to calculate the average of n numbers

Software used – Fedora (Linux)

Theory – Shell scripting in Linux allows performing arithmetic operations using loops and conditional statements. In this program, the user specifies how many numbers to enter. Each number is read inside a loop, and their sum is calculated. Finally, the sum is divided by the total count to compute the average. This demonstrates user input handling, looping, and arithmetic evaluation in Bash scripting.

Source code –

```
echo "How many numbers do you want to enter?"
read n

sum=0

echo "Enter $n numbers: "
for (( i=1; i<=n; i++ ))
do
    read num
    sum=$((sum + num))
done

average=$((sum / n))

echo "The average of $n numbers is: $average"
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# ./average.sh
How many numbers do you want to enter?
5
Enter 5 numbers:
10
20
30
40
50
The average of 5 numbers is: 30
```

PROGRAM – 4(i)

AIM – Write a Script to print Fibonacci series up to n terms

Software used – Fedora (Linux)

Theory – The Fibonacci sequence is a series of numbers where each term is the sum of the two preceding ones, starting with 0 and 1. In this shell script, the user specifies the number of terms to be generated. Two variables a and b are initialized to represent the first two numbers of the sequence. A for loop iterates n times, printing the current number (a), then updating values using arithmetic expansion (`next=$((a+b))`). The values of a and b are shifted forward for each iteration. This process continues until the required number of terms is displayed. The script demonstrates basic concepts of input handling, variables, loops, and arithmetic operations in shell programming.

Source code –

```
echo "Enter the number of terms: "
read n

a=0
b=1

echo "Fibonacci sequence up to $n terms:"

for ((i=0; i<n; i++))
do
    echo -n "$a "
    next=$((a + b))
    a=$b
    b=$next
done

echo
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# vi kp.sh
root@fedora:/home/lab1137-33/Desktop# ./kp.sh
Enter the number of terms:
7
Fibonacci sequence up to 7 terms:
0 1 1 2 3 5 8
```

PROGRAM - 4(ii)

AIM – Write a Script to calculate the factorial of a given number.

Software used – Fedora (Linux)

Theory – The factorial of a number n (denoted as $n!$) is the product of all positive integers from 1 to n . For example, $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$. In this shell script, the user enters a number, and a loop runs from 1 to that number, multiplying each value with a variable factorial. Initially, factorial is set to 1 (since multiplying by 1 does not change the result). At the end of the loop, the factorial value is displayed. This script demonstrates the use of loops, arithmetic expansion, and variables in shell programming.

Source code –

```
echo "Enter a number:"
read num

factorial=1

for ((i=1; i<=num; i++))
do
    factorial=$((factorial * i))
done

echo "The factorial of $num is: $factorial"
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# vi kp.sh
root@fedora:/home/lab1137-33/Desktop# ./kp.sh
Enter a number:
5
The factorial of 5 is: 120

root@fedora:/home/lab1137-33/Desktop# ./kp.sh
Enter a number:
7
The factorial of 7 is: 5040
```

PROGRAM - 4(iii)

AIM – Write a Script to calculate sum of digits of given numbers

Software used – Fedora (Linux)

Theory – The sum of digits of a number is obtained by repeatedly extracting each digit and adding it together. In this script, the last digit is extracted using the modulus operator ($\text{num} \% 10$). This digit is added to a variable sum, and the number is reduced by removing the last digit using integer division ($\text{num} / 10$). The process continues in a while loop until the number becomes zero. Finally, the script prints the total sum of digits. This program demonstrates the use of loops, arithmetic expansion, and modular arithmetic in shell scripting

Source code –

```
echo "Enter a number: "
read num

sum=0
while [ $num -gt 0 ]
do
    digit=$((num % 10))
    sum=$((sum + digit))
    num=$((num / 10))
done

echo "The sum of the digits is: $sum"
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# vi kp.sh
root@fedora:/home/lab1137-33/Desktop# ./kp.sh
Enter a number:
1234
The sum of the digits is: 10

root@fedora:/home/lab1137-33/Desktop# ./kp.sh
Enter a number:
56789
The sum of the digits is: 35
```

PROGRAM - 4(iv)

AIM – Write a Script to check whether the given string is a palindrome or not

Software used – Fedora (Linux)

Theory – A palindrome is a string that reads the same forwards and backwards (e.g., madam, racecar). This script removes spaces and converts the input string to lowercase for uniform comparison. It then uses the rev command to reverse the string and compares it with the cleaned string using [... = ...]. If both match, the string is a palindrome; otherwise, it is not. This script demonstrates string manipulation, command substitution, and conditional checking in shell scripting.

Source code –

```
echo "Enter a string: "
read str

clean_str=$(echo "$str" | tr -d ' ' | tr '[:upper:]' '[:lower:]')

reversed_str=$(echo "$clean_str" | rev)

if [ "$clean_str" = "$reversed_str" ]; then
    echo "$str is a palindrome."
else
    echo "$str is not a palindrome."
fi
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# vi kp.sh
root@fedora:/home/lab1137-33/Desktop# ./kp.sh
Enter a string:
madam
madam is a palindrome.

root@fedora:/home/lab1137-33/Desktop# ./kp.sh
Enter a string:
hello
hello is not a palindrome.

root@fedora:/home/lab1137-33/Desktop# ./kp.sh
Enter a string:
Race Car
Race Car is a palindrome.
```

PROGRAM – 5

AIM – Write a program to implement CPU Scheduling for first come first serve

Software used – Fedora (Linux)

Theory – First Come First Serve (FCFS) is the simplest CPU scheduling algorithm where processes are scheduled in the order they arrive in the queue, following a non-preemptive, first-in-first-out approach; the process arriving first is executed first and continues running until completion without interruption, which makes the algorithm easy to implement but can lead to longer average waiting times, especially when long processes delay the execution of shorter ones

Source code –

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

typedef struct {
    int pid, burst_time, waiting_time, turnaround_time;
} Process;

void print_table(Process p[], int n);
void print_gantt_chart(Process p[], int n);

int main() {
    Process p[MAX];
    int i, n;
    int sum_waiting_time = 0, sum_turnaround_time = 0;

    printf("Enter total number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");
    for (i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("P[%d] : ", i + 1);
        scanf("%d", &p[i].burst_time);
        p[i].waiting_time = 0;
        p[i].turnaround_time = 0;
    }
    p[0].waiting_time = 0;
    p[0].turnaround_time = p[0].burst_time;
```

```

    for (i = 1; i < n; i++) {
        p[i].waiting_time = p[i - 1].waiting_time + p[i - 1].burst_time;
        p[i].turnaround_time = p[i].waiting_time + p[i].burst_time;
    }

    for (i = 0; i < n; i++) {
        sum_waiting_time += p[i].waiting_time;
        sum_turnaround_time += p[i].turnaround_time;
    }

    puts("\nProcess Table:");
    print_table(p, n);
    printf("\nTotal Waiting Time    : %d\n", sum_waiting_time);
    printf("Average Waiting Time    : %.2lf\n", (double)sum_waiting_time / n);
    printf("Total Turnaround Time    : %d\n", sum_turnaround_time);
    printf("Average Turnaround Time : %.2lf\n", (double)sum_turnaround_time / n);

    puts("\n GANTT CHART ");
    puts(" ***** ");
    print_gantt_chart(p, n);
    return 0;
}

void print_table(Process p[], int n) {
    int i;
    printf("+-----+-----+-----+-----+\n");
    printf("| PID | Burst Time | Waiting Time | Turnaround Time |\n");
    printf("+-----+-----+-----+-----+\n");
    for (i = 0; i < n; i++) {
        printf("| %3d | %10d | %12d | %15d |\n",
            p[i].pid, p[i].burst_time, p[i].waiting_time, p[i].turnaround_time);
    }
    printf("+-----+-----+-----+-----+\n");
}

void print_gantt_chart(Process p[], int n) {
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = 0; j < p[i].burst_time; j++) printf("--");
        printf(" ");
    }
    printf("\n");

    for (i = 0; i < n; i++) {

```

```

    printf("|");
    for (j = 0; j < p[i].burst_time - 1; j++) printf(" ");
    printf("P%d", p[i].pid);
    for (j = 0; j < p[i].burst_time - 1; j++) printf(" ");
}
printf("\n");

for (i = 0; i < n; i++) {
    for (j = 0; j < p[i].burst_time; j++) printf("--");
    printf(" ");
}
printf("\n");

printf("0");
for (i = 0; i < n; i++) {
    for (j = 0; j < p[i].burst_time; j++) printf(" ");
    if (p[i].turnaround_time > 9) printf("\b");
    printf("%d", p[i].turnaround_time);
}
printf("\n");
}

```

Output: -

```

root@fedora:/home/lab1137-33/Desktop# vi five.c
root@fedora:/home/lab1137-33/Desktop# gcc five.c
root@fedora:/home/lab1137-33/Desktop# ./a.out
Enter total number of processes: 3
Enter burst time for each process:
P[1] : 5
P[2] : 3
P[3] : 8

Process Table:
+-----+-----+-----+-----+
| PID | Burst Time | Waiting Time | Turnaround Time |
+-----+-----+-----+-----+
| 1 | 5 | 0 | 5 |
| 2 | 3 | 5 | 8 |
| 3 | 8 | 8 | 16 |
+-----+-----+-----+-----+

Total Waiting Time : 13
Average Waiting Time : 4.33
Total Turnaround Time : 29
Average Turnaround Time : 9.67

GANTT CHART
*****
-----
| P1 | P2 | P3 |
-----
0 5 8 16

root@fedora:/home/lab1137-33/Desktop#

```


PROGRAM - 6

AIM – Write a program to implement CPU Scheduling for shortest job first

Software used – Fedora (Linux)

Theory – Shortest Job First (SJF) CPU scheduling algorithm selects the waiting process with the smallest burst time for execution, minimizing average waiting time compared to other strategies; either preemptive or non-preemptive, this greedy approach can cause starvation for longer processes if short jobs continually arrive, and it requires estimation of process burst times which may not always be possible in real systems.

Source code –

```
#include <stdio.h>

struct Process {
    int pid, burstTime, waitingTime, turnaroundTime;
};

void sortByBurst(struct Process p[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (p[i].burstTime > p[j].burstTime) {
                struct Process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

int main() {
    int n, i;
    struct Process p[20];
    int totalWT = 0, totalTAT = 0;

    printf("Enter total process: ");
    scanf("%d", &n);
```

```

printf("Enter burst time for each process:\n");
for (i = 0; i < n; i++) {
    p[i].pid = i + 1;
    printf("P[%d]: ", p[i].pid);
    scanf("%d", &p[i].burstTime);
}
sortByBurst(p, n);

p[0].waitingTime = 0;
p[0].turnaroundTime = p[0].burstTime;

for (i = 1; i < n; i++) {
    p[i].waitingTime = p[i - 1].waitingTime + p[i - 1].burstTime;
    p[i].turnaroundTime = p[i].waitingTime + p[i].burstTime;
}

for (i = 0; i < n; i++) {
    totalWT += p[i].waitingTime;
    totalTAT += p[i].turnaroundTime;
}

printf("\nGantt Chart:\n ");
for (i = 0; i < n; i++) {
    for (int j = 0; j < p[i].burstTime; j++) printf("--");
    printf(" ");
}
printf("\n|");
for (i = 0; i < n; i++) {
    for (int j = 0; j < p[i].burstTime - 1; j++) printf(" ");
    printf("P%d", p[i].pid);
    for (int j = 0; j < p[i].burstTime - 1; j++) printf(" ");
    printf("|");
}
printf("\n ");
for (i = 0; i < n; i++) {

```

```

        for (int j = 0; j < p[i].burstTime; j++) printf("--");
        printf(" ");
    }
    printf("\n");

    int time = 0;
    printf("%d", time);
    for (i = 0; i < n; i++) {
        time += p[i].burstTime;
        printf("%d", p[i].burstTime * 2, time);
    }
    printf("\n");

    printf("\nTotal Waiting Time: %d", totalWT);
    printf("\nAverage Waiting Time: %.2f", (double)totalWT / n);
    printf("\nTotal Turnaround Time: %d", totalTAT);
    printf("\nAverage Turnaround Time: %.2f", (double)totalTAT / n);
    return 0;
}

```

Output: -

```

root@fedora:/home/lab1137-33/Desktop# vi six.c
root@fedora:/home/lab1137-33/Desktop# gcc six.c
root@fedora:/home/lab1137-33/Desktop# ./a.out
Enter total process: 4
Enter burst time for each process:
P[1]: 7
P[2]: 3
P[3]: 4
P[4]: 6

Gantt Chart:
-----
|  P2  |  P3  |      P4      |      P1      |
-----
0      3      7      13      20

Total Waiting Time: 23
Average Waiting Time: 5.75
Total Turnaround Time: 43
Average Turnaround Time: 10.75

root@fedora:/home/lab1137-33/Desktop#

```

PROGRAM - 7

AIM – Write a Program to perform Priority Scheduling

Software used – Fedora (Linux)

Theory – Priority Scheduling algorithm assigns a priority to each process and the CPU selects the process with the highest priority for execution next, meaning processes with lower numeric values (indicating higher priority) are scheduled before others; if multiple processes have the same priority, they are scheduled by arrival time, and this method effectively reduces waiting time for critical tasks but can cause starvation for those with lower priority.

Source code –

```
#include <stdio.h>
#include <limits.h>

struct Process {
    int pid, arrivalTime, burstTime, priority;
    int remainingTime, completionTime;
    int waitingTime, turnaroundTime;
};

void calculateTimes(struct Process processes[], int n) {
    int complete = 0, currentTime = 0;
    int highestPriority = -1, finishTime;
    int minPriority, check;

    while (complete != n) {
        minPriority = INT_MAX;
        check = 0;

        for (int i = 0; i < n; i++) {
            if ((processes[i].arrivalTime <= currentTime) &&
                (processes[i].priority < minPriority) &&
                (processes[i].remainingTime > 0)) {
                minPriority = processes[i].priority;
                highestPriority = i;
            }
        }
    }
}
```

```

        check = 1;
    }
}
if (check == 0) {
    currentTime++;
    continue;
}
processes[highestPriority].remainingTime--;

if (processes[highestPriority].remainingTime == 0) {
    complete++;
    finishTime = currentTime + 1;
    processes[highestPriority].completionTime = finishTime;
    processes[highestPriority].turnaroundTime =
        finishTime - processes[highestPriority].arrivalTime;
    processes[highestPriority].waitingTime =
        processes[highestPriority].turnaroundTime -
        processes[highestPriority].burstTime;

    if (processes[highestPriority].waitingTime < 0)
        processes[highestPriority].waitingTime = 0;
}
currentTime++;
}
}

void printTable(struct Process processes[], int n) {
    printf("\nPID\tAT\tBT\tPT\tCT\tWT\tTAT\n");
    int totalTurnaroundTime = 0, totalWaitingTime = 0;

    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            processes[i].pid, processes[i].arrivalTime,
            processes[i].burstTime, processes[i].priority,
            processes[i].completionTime,

```

```

        processes[i].waitingTime, processes[i].turnaroundTime);

    totalTurnaroundTime += processes[i].turnaroundTime;
    totalWaitingTime += processes[i].waitingTime;
}

printf("\nTotal Waiting Time: %d\n", totalWaitingTime);
printf("Average Waiting Time: %.2f\n", (float)totalWaitingTime / n);
printf("Total Turnaround Time: %d\n", totalTurnaroundTime);
printf("Average Turnaround Time: %.2f\n",
       (float)totalTurnaroundTime / n);
}

void printGanttChart(struct Process processes[], int n) {
    printf("\nGantt Chart:\n");
    for (int i = 0; i < n; i++) {
        printf("| P%d  ", processes[i].pid);
    }

    printf("\n");

    int time = 0;
    printf("%d", time);
    for (int i = 0; i < n; i++) {
        time = processes[i].completionTime;
        printf("\t%d", time);
    }
    printf("\n");
}

int main() {
    int n;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    struct Process processes[n];

```

```

for (int i = 0; i < n; i++) {
    processes[i].pid = i + 1;
    printf("Enter arrival time, burst time, and priority for P%d: ",
        processes[i].pid);
    scanf("%d%d%d", &processes[i].arrivalTime,
        &processes[i].burstTime, &processes[i].priority);
    processes[i].remainingTime = processes[i].burstTime;
}

calculateTimes(processes, n);
printTable(processes, n);
printGanttChart(processes, n);
return 0;
}

```

Output: -

```

root@fedora:/home/lab1137-33/Desktop# vi seven.c
root@fedora:/home/lab1137-33/Desktop# gcc seven.c
root@fedora:/home/lab1137-33/Desktop# ./a.out
Enter the number of processes: 3
Enter arrival time, burst time, and priority for P1: 0 5 2
Enter arrival time, burst time, and priority for P2: 1 3 1
Enter arrival time, burst time, and priority for P3: 2 4 3

PID    AT    BT    PT    CT    WT    TAT
P1      0     5     2     8     3     8
P2      1     3     1     4     0     3
P3      2     4     3    12     6    10

Total Waiting Time: 9
Average Waiting Time: 3.00
Total Turnaround Time: 21
Average Turnaround Time: 7.00

Gantt Chart:
|  P1  |  P2  |  P3  |
0      8      4      12

root@fedora:/home/lab1137-33/Desktop#

```

PROGRAM - 8

AIM – Write a program to implement CPU Scheduling for Round Robin

Software used – Fedora (Linux)

Theory – Round Robin (RR) CPU Scheduling assigns each process a fixed time quantum during which it can execute, cycling through all processes in the ready queue to ensure fair CPU allocation; if a process does not finish in its time slice, it is preempted and placed at the end of the queue to wait for its next turn, enabling time-sharing and preventing starvation.

Source code –

```
echo -n "Enter number of processes: "
read n
echo -n "Enter time quantum: "
read tq

for ((i=0; i<n; i++)); do
    echo -n "Enter arrival time & burst time for process P$((i+1)): "
    read at[i] bt[i]
    p[i]=$i
    temp[i]={bt[i]}
    tat[i]=0
    wt[i]=0
    ct[i]=0
done

gantt_process=()
gantt_time=()
time=0
remain=$n

while ((remain != 0)); do
    executed=0
    for ((i=0; i<n; i++)); do
        if ((temp[i] > 0 && at[i] <= time)); then
            gantt_process+=({p[i]})
```



```

gantt_time+=($time)

if ((temp[i] <= tq)); then
    time=$((time + temp[i]))
    ct[$i]=$time
    tat[$i]=$((ct[i] - at[i]))
    wt[$i]=$((tat[i] - bt[i]))
    temp[$i]=0
    remain=$((remain - 1))
else
    temp[$i]=$((temp[i] - tq))
    time=$((time + tq))
fi
executed=1
fi
done

if ((executed == 0)); then
    time=$((time + 1))
fi
done

gantt_time+=($time)

echo ""
echo "+-----+---+---+---+---+---+"
echo "| Process | AT | BT | CT | TAT | WT |"
echo "+-----+---+---+---+---+---+"

total_tat=0
total_wt=0
for ((i=0; i<n; i++)); do
    echo "| P$((i+1)) | ${at[$i]} | ${bt[$i]} | ${ct[$i]} | ${tat[$i]} | ${wt[$i]} |"
    total_tat=$((total_tat + tat[i]))
    total_wt=$((total_wt + wt[i]))
done

```

```

done
echo "+-----+---+---+---+---+---+"
avg_tat=$(echo "scale=2; $total_tat / $n" | bc)
avg_wt=$(echo "scale=2; $total_wt / $n" | bc)

echo "Total Waiting Time : $total_wt"
echo "Average Waiting Time : $avg_wt"
echo "Total Turnaround Time: $total_tat"
echo "Average Turnaround : $avg_tat"

echo -e "\nGantt Chart:"
for ((i=0; i<${#gantt_process[@]}; i++)); do
    echo -n "| P${(gantt_process[i] + 1)} "
done
echo "|"

for ((i=0; i<${#gantt_time[@]}; i++)); do
    echo -n "${gantt_time[i]} "
done
echo ""

```

Output: -

```

root@fedora:/home/lab1137-33/Desktop# vi eight.sh
root@fedora:/home/lab1137-33/Desktop# chmod +x eight.sh
root@fedora:/home/lab1137-33/Desktop# ./eight.sh
Enter number of processes: 4
Enter time quantum: 3
Enter arrival time & burst time for process P1: 0 8
Enter arrival time & burst time for process P2: 1 5
Enter arrival time & burst time for process P3: 2 10
Enter arrival time & burst time for process P4: 3 4

+-----+---+---+---+---+---+
| Process | AT | BT | CT | TAT | WT |
+-----+---+---+---+---+---+
| P1      | 0  | 8  | 22 | 22  | 14 |
| P2      | 1  | 5  | 17 | 16  | 11 |
| P3      | 2  | 10 | 27 | 25  | 15 |
| P4      | 3  | 4  | 23 | 20  | 16 |
+-----+---+---+---+---+---+

Total Waiting Time : 56
Average Waiting Time : 14.00
Total Turnaround Time: 83
Average Turnaround : 20.75

Gantt Chart:
| P1 | P2 | P3 | P4 | P1 | P2 | P3 | P4 | P1 | P3 |
0   3   6   9   12  15  18  21  24  27

root@fedora:/home/lab1137-33/Desktop#

```

PROGRAM – 9 (i)

AIM – Write a program for page replacement policy using LRU

Software used – Fedora (Linux)

Theory – The Least Recently Used (LRU) algorithm replaces the page that has not been used for the longest period. This policy is based on the assumption that pages that have been recently used are likely to be used again, and pages that have not been used in a long time are least likely to be used again. To implement this, the system must keep track of when each page was last used. When a page fault occurs and the frames are full, the page with the oldest "last used" timestamp is selected as the victim

Source code –

```
#!/bin/bash
echo "Enter number of frames:"
read nf
echo "Enter reference string (space separated):"
read -a ref

declare -a frame
declare -a last_used
time=0
pf=0
printf "\n%-8s %-8s %-20s %-10s\n" "Step" "Page" "Frames" "Status"
echo "-----"

for ((i=0; i<${#ref[@]}; i++))
do
    page=${ref[i]}
    found=0

    for ((j=0; j<nf; j++))
    do
        if [[ ${frame[j]} -eq $page ]]
        then
            found=1
            last_used[j]=$time
            status="Hit"
            break
        fi
    done
    if [[ $found -eq 0 ]]
    then
```

```

if [[ ${#frame[@]} -lt nf ]]
then
    frame[${#frame[@]}]=$page
    last_used[${#frame[@]}-1]=$time
else
    lru_index=0
    min_time=${last_used[0]}

    for ((j=1; j<nf; j++))
    do
        if [[ ${last_used[j]} -lt $min_time ]]
        then
            min_time=${last_used[j]}
            lru_index=j
        fi
    done

    frame[$lru_index]=$page
    last_used[$lru_index]=$time
fi
((pf++))
status="Page Fault"
fi

printf "%-8s %-8s %-20s %-10s\n" "$((i+1))" "$page" "${frame[*]}" "$status"
((time++))
done
echo "-----"
echo "Total Page Faults (LRU): $pf"

```

Output: -

```

root@fedora:/home/lab1137-33/Desktop# vim lru.sh
root@fedora:/home/lab1137-33/Desktop# ls
lru.sh
root@fedora:/home/lab1137-33/Desktop# chmod +x lru.sh
root@fedora:/home/lab1137-33/Desktop# ./lru.sh
Enter number of frames: 3
Enter reference string (space separated): 7 0 1 2 0 3 0 4 2 3 0 3

Step    Page    Frames          Status
-----
1        7        7              Page Fault
2        0        7 0           Page Fault
3        1        7 0 1         Page Fault
4        2        2 0 1         Page Fault
5        0        2 0 1         Hit
6        3        2 0 3         Page Fault
7        0        2 0 3         Hit
8        4        4 0 3         Page Fault
9        2        4 0 2         Page Fault
10       3        4 3 2         Page Fault
11       0        0 3 2         Page Fault
12       3        0 3 2         Hit
13       2        0 3 2         Hit
-----
Total Page Faults (LRU): 9

```

PROGRAM – 9 (ii)

AIM – Write a program for page replacement policy using FIFO

Software used – Fedora (Linux)

Theory – The First-In, First-Out (FIFO) algorithm is the simplest page replacement policy. When a page fault occurs and all frames are full, the page that has been in memory the longest (the "oldest" page) is selected as the victim to be replaced. This is implemented using a simple queue or a circular index that points to the next frame to be replaced.

Source code –

```
#!/bin/bash

echo "Enter number of frames:"
read nf
echo "Enter reference string (space separated):"
read -a ref

declare -a frame
pf=0
index=0

for ((i=0; i < ${#ref[@]}; i++)); do
    page=${ref[i]}
    found=0

    for ((j=0; j < ${#frame[@]}; j++)); do
        if [[ ${frame[j]} -eq $page ]]; then
            found=1
            break
        fi
    done

    if [[ $found -eq 0 ]]; then
        if [[ ${#frame[@]} -lt nf ]]; then
            frame[${#frame[@]}]=$page
        else
            frame[$index]=$page
            index=$((index + 1) % nf)
        fi
        ((pf++))
    fi
done
```

```
    echo "Step $((i + 1)) [Page: $page] => Frames: ${frame[*]}"  
done
```

```
echo "Total Page Faults (FIFO): $pf"
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# vim fifo.sh  
root@fedora:/home/lab1137-33/Desktop# ls  
fifo.sh  
root@fedora:/home/lab1137-33/Desktop# chmod +x fifo.sh  
root@fedora:/home/lab1137-33/Desktop# ./fifo.sh  
Enter number of frames: 3  
Enter reference string (space separated): 7 0 1 2 0 3 0 4 2 3 0 3  
  
Step 1 [Page: 7] => Frames: 7  
Step 2 [Page: 0] => Frames: 7 0  
Step 3 [Page: 1] => Frames: 7 0 1  
Step 4 [Page: 2] => Frames: 2 0 1  
Step 5 [Page: 0] => Frames: 2 0 1  
Step 6 [Page: 3] => Frames: 2 3 1  
Step 7 [Page: 0] => Frames: 2 3 0  
Step 8 [Page: 4] => Frames: 4 3 0  
Step 9 [Page: 2] => Frames: 4 2 0  
Step 10 [Page: 3] => Frames: 4 2 3  
Step 11 [Page: 0] => Frames: 0 2 3  
Step 12 [Page: 3] => Frames: 0 2 3  
Step 13 [Page: 2] => Frames: 0 2 3  
Total Page Faults (FIFO): 10
```

PROGRAM – 9 (iii)

AIM – Write a program for page replacement policy using Optimal

Software used – Fedora (Linux)

Theory – The Optimal (OPT) page replacement algorithm has the lowest possible page fault rate and is used as a benchmark for other algorithms. When a page fault occurs and frames are full, the Optimal algorithm replaces the page that will *not be used for the longest period in the future*. This requires the algorithm to have knowledge of the entire future reference string, which is impossible in a real system. However, it is useful for comparison and analysis

Source code –

```
#!/bin/bash

echo "Enter number of frames:"
read nf
echo "Enter reference string (space separated):"
read -a ref

declare -a frame
pf=0

for ((i=0; i<${#ref[@]}; i++)); do
    page=${ref[i]}
    found=0

    for ((j=0; j<${#frame[@]}; j++)); do
        if [[ ${frame[j]} -eq $page ]]; then
            found=1
            break
        fi
    done

    if [[ $found -eq 0 ]]; then
        if [[ ${#frame[@]} -lt nf ]]; then
            # If frame has space, insert the page
            frame[${#frame[@]}]=$page
        else
            farthest=-1
            replace_index=-1

            for ((j=0; j<nf; j++)); do
```

```

k=$((i + 1))

while [[ $k -lt ${#ref[@]} && ${ref[k]} -ne ${frame[j]} ]]; do
    ((k++))
done

if [[ $k -gt $farthest ]]; then
    farthest=$k
    replace_index=$j
fi
done

frame[$replace_index]=$page
fi

((pf++))
fi

echo "Step $((i + 1)) [Page: $page] => Frames: ${frame[@]}"
done

echo "Total Page Faults (Optimal): $pf"

```

Output: -

```

root@fedora:/home/lab1137-33/Desktop# vim op.sh
root@fedora:/home/lab1137-33/Desktop# ls
op.sh
root@fedora:/home/lab1137-33/Desktop# chmod +x op.sh
root@fedora:/home/lab1137-33/Desktop# ./op.sh
Enter number of frames: 3
Enter reference string (space separated): 7 0 1 2 0 3 0 4 2 3 0 3 2
Step 1 [Page: 7] => Frames: 7
Step 2 [Page: 0] => Frames: 7 0
Step 3 [Page: 1] => Frames: 7 0 1
Step 4 [Page: 2] => Frames: 2 0 1
Step 5 [Page: 0] => Frames: 2 0 1
Step 6 [Page: 3] => Frames: 2 0 3
Step 7 [Page: 0] => Frames: 2 0 3
Step 8 [Page: 4] => Frames: 2 0 4
Step 9 [Page: 2] => Frames: 2 0 4
Step 10 [Page: 3] => Frames: 3 0 4
Step 11 [Page: 0] => Frames: 3 0 4
Step 12 [Page: 3] => Frames: 3 0 4
Step 13 [Page: 2] => Frames: 3 0 2
Total Page Faults (Optimal): 9

```


PROGRAM - 10

AIM – Write a program to implement first fit, best fit and worst fit algorithm for memory management

Software used – Fedora (Linux)

Theory – Memory management algorithms determine how to allocate memory blocks to processes. This program simulates three "contiguous allocation" strategies: • First Fit: Allocates the *first* hole (memory block) that is big enough to accommodate the process. • Best Fit: Allocates the *smallest* hole that is big enough. This requires searching the entire list of holes. • Worst Fit: Allocates the *largest* hole. This also requires searching the entire list. The program will take a set of block sizes and process sizes and show the allocation for each strategy.

Source code –

```
#include <stdio.h>
#include <string.h>

// Function for First Fit
void firstFit(int blocks[], int m, int processes[], int n) {
    int allocation[n];
    memset(allocation, -1, sizeof(allocation));

    int tempBlocks[m];
    for (int i = 0; i < m; i++)
        tempBlocks[i] = blocks[i];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (tempBlocks[j] >= processes[i]) {
                allocation[i] = j;
                tempBlocks[j] -= processes[i];
                break;
            }
        }
    }

    printf("\n--- First Fit Allocation ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t", i + 1, processes[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
    }
}
```

```

        else
            printf("Not Allocated\n");
    }
}

```

// Function for Best Fit

```

void bestFit(int blocks[], int m, int processes[], int n) {
    int allocation[n];
    memset(allocation, -1, sizeof(allocation));

    int tempBlocks[m];
    for (int i = 0; i < m; i++)
        tempBlocks[i] = blocks[i];

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (tempBlocks[j] >= processes[i]) {
                if (bestIdx == -1 || tempBlocks[j] < tempBlocks[bestIdx]) {
                    bestIdx = j;
                }
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            tempBlocks[bestIdx] -= processes[i];
        }
    }
}

```

```

printf("\n--- Best Fit Allocation ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processes[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

```

// Function for Worst Fit

```

void worstFit(int blocks[], int m, int processes[], int n) {
    int allocation[n];

```

```
memset(allocation, -1, sizeof(allocation));
```

```
int tempBlocks[m];
```

```
for (int i = 0; i < m; i++)
```

```
    tempBlocks[i] = blocks[i];
```

```
for (int i = 0; i < n; i++) {
```

```
    int worstIdx = -1;
```

```
    for (int j = 0; j < m; j++) {
```

```
        if (tempBlocks[j] >= processes[i]) {
```

```
            if (worstIdx == -1 || tempBlocks[j] > tempBlocks[worstIdx]) {
```

```
                worstIdx = j;
```

```
            }
```

```
        }
```

```
    }
```

```
    if (worstIdx != -1) {
```

```
        allocation[i] = worstIdx;
```

```
        tempBlocks[worstIdx] -= processes[i];
```

```
    }
```

```
}
```

```
printf("\n--- Worst Fit Allocation ---\n");
```

```
printf("Process No.\tProcess Size\tBlock No.\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("%d\t\t%d\t\t", i + 1, processes[i]);
```

```
    if (allocation[i] != -1)
```

```
        printf("%d\n", allocation[i] + 1);
```

```
    else
```

```
        printf("Not Allocated\n");
```

```
}
```

```
}
```

```
int main() {
```

```
    int m, n;
```

```
    int blocks[50], processes[50];
```

```
    printf("Enter number of memory blocks: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter size of each block:\n");
```

```
    for (int i = 0; i < m; i++)
```

```
        scanf("%d", &blocks[i]);
```

```

printf("Enter number of processes: ");
scanf("%d", &n);

printf("Enter size of each process:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &processes[i]);

firstFit(blocks, m, processes, n);
bestFit(blocks, m, processes, n);
worstFit(blocks, m, processes, n);

return 0;
}

```

Output: -

```

root@fedora:/home/lab1137-33/Desktop# vim memory_fit.c
root@fedora:/home/lab1137-33/Desktop# ls
memory_fit.c
root@fedora:/home/lab1137-33/Desktop# gcc memory_fit.c -o memory_fit
root@fedora:/home/lab1137-33/Desktop# ./memory_fit
Enter number of memory blocks: 5
Enter size of each block:
100
500
200
300
600
Enter number of processes: 4
Enter size of each process:
212
417
112
426

--- First Fit Allocation ---
Process No.    Process Size    Block No.
1              212           2
2              417           5
3              112           2
4              426          Not Allocated

--- Best Fit Allocation ---
Process No.    Process Size    Block No.
1              212           3
2              417           5
3              112           1
4              426          Not Allocated

--- Worst Fit Allocation ---
Process No.    Process Size    Block No.
1              212           5
2              417           2
3              112           5
4              426          Not Allocated

```

PROGRAM - 11

AIM – Write a program to implement reader/writer problem using semaphore.

Software used – Fedora (Linux)

Theory – The Reader-Writer problem is a classic synchronization problem. It models a scenario where multiple processes (readers) can read a shared resource simultaneously, but only one process (writer) can modify it at a time. While a writer is active, no other reader or writer can access the resource.

This solution uses two semaphores and a counter:

- `rw\_mutex` (or `wrt`): A binary semaphore that controls write access. It is locked by any writer, and by the *first* reader.
- `mutex`: A binary semaphore that provides exclusive access to the `read\_count` variable.
- `read\_count`: An integer tracking the number of active readers

Source code –

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

sem_t mutex, rw_mutex;
int read_count = 0;
int shared_data = 1;

void *reader(void *arg) {
    int id = *(int *)arg;

    sem_wait(&mutex);
    read_count++;
    if (read_count == 1) {
        sem_wait(&rw_mutex);
    }
    sem_post(&mutex);

    printf("Reader %d is reading data: %d\n", id, shared_data);
    sleep(1);

    sem_wait(&mutex);
    read_count--;
    if (read_count == 0) {
```

```

        sem_post(&rw_mutex);
    }
    sem_post(&mutex);

    printf("Reader %d has finished reading.\n", id);
    return NULL;
}

void *writer(void *arg) {
    int id = *(int *)arg;
    sem_wait(&rw_mutex);

    shared_data = id + 10;
    printf("Writer %d is writing data: %d\n", id, shared_data);
    sleep(2);

    sem_post(&rw_mutex);

    printf("Writer %d has finished writing.\n", id);
    return NULL;
}

int main() {
    int num_readers = 5;
    int num_writers = 2;

    pthread_t readers[num_readers];
    pthread_t writers[num_writers];
    int reader_ids[num_readers];
    int writer_ids[num_writers];

    sem_init(&mutex, 0, 1);
    sem_init(&rw_mutex, 0, 1);

    for (int i = 0; i < num_readers; i++) {
        reader_ids[i] = i + 1;
        pthread_create(&readers[i], NULL, reader, &reader_ids[i]);
    }

    for (int i = 0; i < num_writers; i++) {
        writer_ids[i] = i + 1;
        pthread_create(&writers[i], NULL, writer, &writer_ids[i]);
    }
}

```

```
    for (int i = 0; i < num_readers; i++) {  
        pthread_join(readers[i], NULL);  
    }  
    for (int i = 0; i < num_writers; i++) {  
        pthread_join(writers[i], NULL);  
    }  
  
    sem_destroy(&mutex);  
    sem_destroy(&rw_mutex);  
  
    return 0;  
}
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# vim readers_writers.c  
root@fedora:/home/lab1137-33/Desktop# ls  
readers_writers.c  
root@fedora:/home/lab1137-33/Desktop# gcc readers_writers.c -o readers_writers  
root@fedora:/home/lab1137-33/Desktop# ./readers_writers  
Reader 1 is reading data: 1  
Reader 2 is reading data: 1  
Reader 3 is reading data: 1  
Reader 4 is reading data: 1  
Reader 5 is reading data: 1  
Reader 1 has finished reading.  
Reader 2 has finished reading.  
Reader 3 has finished reading.  
Reader 4 has finished reading.  
Reader 5 has finished reading.  
Writer 1 is writing data: 11  
Writer 1 has finished writing.  
Writer 2 is writing data: 12  
Writer 2 has finished writing.
```

PROGRAM - 12

AIM – Write a program to implement Producer-Consumer problem using semaphores.

Software used – Fedora (Linux)

Theory –The Producer-Consumer problem (or Bounded Buffer problem) involves two types of processes: Producers, who create data and add it to a shared buffer, and Consumers, who remove and use data from the buffer. The problem is to ensure that the producer doesn't add data to a full buffer and the consumer doesn't try to remove data from an empty buffer.

Source code –

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    int item;
    for (int i = 0; i < 5; i++) {
        item = i + 1;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer produced item %d at buffer[%d]\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        sleep(1);

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void *consumer(void *arg) {
    int item;
    for (int i = 0; i < 5; i++) {
```



```

        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer consumed item %d from buffer[%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        sleep(1);

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Output: -

```

root@fedora:/home/lab1137-33/Desktop# vim producer_consumer.c
root@fedora:/home/lab1137-33/Desktop# ls
producer_consumer.c
root@fedora:/home/lab1137-33/Desktop# gcc producer_consumer.c -o producer_consumer
root@fedora:/home/lab1137-33/Desktop# ./producer_consumer
Producer produced item 1 at buffer[0]
Consumer consumed item 1 from buffer[0]
Producer produced item 2 at buffer[1]
Consumer consumed item 2 from buffer[1]
Producer produced item 3 at buffer[2]
Consumer consumed item 3 from buffer[2]
Producer produced item 4 at buffer[3]
Consumer consumed item 4 from buffer[3]
Producer produced item 5 at buffer[4]
Consumer consumed item 5 from buffer[4]

```

PROGRAM – 13

AIM – Write a program to implement Banker's algorithm for deadlock avoidance.

Software used – Fedora (Linux)

Theory – The Banker's Algorithm is a deadlock avoidance algorithm. It models a bank that has a fixed amount of capital (resources) and a set of clients (processes) to whom it has granted credit lines (maximum resource needs). The algorithm checks if granting a loan (resource request) to a client will leave the bank in a "safe state" — a state where there is some sequence in which all clients can eventually finish their work.

It uses several data structures:

- Allocation: Resources currently allocated to each process.
- Max: Maximum resources each process might request.
- Available: Resources currently unallocated.
- Need: 'Max - Allocation'. This is what each process still needs to complete.

Source code –

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], avail[m];
    int need[n][m];

    printf("Enter the Allocation matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter the Max matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter the Available resources vector:\n");
```

```

for (j = 0; j < m; j++)
    scanf("%d", &avail[j]);

for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        need[i][j] = max[i][j] - alloc[i][j];

int finish[n], work[m], safeSeq[n];
int count = 0;

for (j = 0; j < m; j++)
    work[j] = avail[j];
for (i = 0; i < n; i++)
    finish[i] = 0;

while (count < n) {
    int found = 0;
    for (i = 0; i < n; i++) {
        if (finish[i] == 0) {
            int canExecute = 1;

            for (j = 0; j < m; j++) {
                if (need[i][j] > work[j]) {
                    canExecute = 0;
                    break;
                }
            }
            if (canExecute) {
                for (k = 0; k < m; k++)
                    work[k] += alloc[i][k];

                finish[i] = 1;
                safeSeq[count++] = i;
                found = 1;
            }
        }
    }
    if (found == 0) {
        printf("\nSystem is in an unsafe state.\n");
        return 0;
    }
}
printf("\nSystem is in a safe state.\n");

```

```
printf("Safe Sequence: ");
for (i = 0; i < n - 1; i++)
    printf("P%d -> ", safeSeq[i]);
printf("P%d\n", safeSeq[n - 1]);

return 0;
}
```

Output: -

```
root@fedora:/home/lab1137-33/Desktop# vim bankers.c
root@fedora:/home/lab1137-33/Desktop# gcc bankers.c -o bankers
root@fedora:/home/lab1137-33/Desktop# ./bankers
Enter number of processes: 5
Enter number of resource types: 3
Enter the Allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter the Max matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter the Available resources vector:
3 3 2

System is in a safe state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

PROGRAM - 14

AIM – Write a C program to implement various File Organization Techniques.

Software used – Fedora (Linux)

Theory – The File organization refers to the way records are stored and accessed within a file. This program simulates two common techniques:

1. Sequential Organization: Records are stored in a specific sequence, one after another. To access a record, you must read all preceding records. It is simple but slow for random access.
2. Indexed Sequential Organization: This method provides a balance. Records are stored sequentially (like in a data file), but a separate index file is maintained. The index file stores keys and their physical addresses (e.g., byte offset) in the data file. To find a record, the program first searches the small index, then directly jumps to the record's location using `fseek()`.

Source code –

```
#include <stdio.h>
#include <stdlib.h>

struct Record {
    int id;
    char name[50];
};

struct Index {
    int id;
    long offset;
};

void createSequential() {
    FILE *fp = fopen("seq.dat", "wb");
    if (!fp) { printf("Error opening file.\n"); return; }

    int n;
    struct Record rec;
    printf("Enter number of records: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter ID for record %d: ", i + 1);
        scanf("%d", &rec.id);
        printf("Enter Name for record %d: ", i + 1);
        scanf("%s", rec.name);
```

```

        fwrite(&rec, sizeof(struct Record), 1, fp);
    }
    fclose(fp);
    printf("Sequential file created.\n");
}

void displaySequential() {
    FILE *fp = fopen("seq.dat", "rb");
    if (!fp) { printf("File not found.\n"); return; }

    struct Record rec;
    printf("\n--- Records in Sequential File ---\n");
    while (fread(&rec, sizeof(struct Record), 1, fp)) {
        printf("ID: %d, Name: %s\n", rec.id, rec.name);
    }
    fclose(fp);
}

void searchSequential() {
    int id, found = 0;
    FILE *fp = fopen("seq.dat", "rb");
    if (!fp) { printf("File not found.\n"); return; }
    printf("Enter ID to search: ");
    scanf("%d", &id);

    struct Record rec;
    while (fread(&rec, sizeof(struct Record), 1, fp)) {
        if (rec.id == id) {
            printf("Record Found: ID: %d, Name: %s\n", rec.id, rec.name);
            found = 1;
            break;
        }
    }
    if (!found) printf("Record not found.\n");
    fclose(fp);
}

void createIndexed() {
    FILE *fpData = fopen("data.dat", "wb");
    FILE *fpIndex = fopen("index.dat", "wb");
    if (!fpData || !fpIndex) { printf("Error creating files.\n"); return; }

    int n;
    struct Record rec;
    struct Index idx;
    printf("Enter number of records: ");

```

```

scanf("%d", &n);

for (int i = 0; i < n; i++) {
    printf("Enter ID for record %d: ", i + 1);
    scanf("%d", &rec.id);
    printf("Enter Name for record %d: ", i + 1);
    scanf("%s", rec.name);
    idx.id = rec.id;
    idx.offset = ftell(fpData);
    fwrite(&rec, sizeof(struct Record), 1, fpData);
    fwrite(&idx, sizeof(struct Index), 1, fpIndex);
}
fclose(fpData);
fclose(fpIndex);
printf("Indexed files created.\n");
}

void searchIndexed() {
    int id, found = 0;
    FILE *fpData = fopen("data.dat", "rb");
    FILE *fpIndex = fopen("index.dat", "rb");
    if (!fpData || !fpIndex) { printf("Files not found.\n"); return; }

    printf("Enter ID to search: ");
    scanf("%d", &id);
    struct Record rec;
    struct Index idx;
    while (fread(&idx, sizeof(struct Index), 1, fpIndex)) {
        if (idx.id == id) {
            fseek(fpData, idx.offset, SEEK_SET);
            fread(&rec, sizeof(struct Record), 1, fpData);
            printf("Record Found: ID: %d, Name: %s\n", rec.id, rec.name);
            found = 1;
            break;
        }
    }
    if (!found) printf("Record not found.\n");
    fclose(fpData);
    fclose(fpIndex);
}

int main() {
    int choice;
    do {
        printf("\n--- File Organization Techniques ---\n");

```

```

printf("1. Sequential File (Create)\n");
printf("2. Sequential File (Display)\n");
printf("3. Sequential File (Search)\n");
printf("4. Indexed File (Create)\n");
printf("5. Indexed File (Search)\n");
printf("0. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice) {
    case 1: createSequential(); break;
    case 2: displaySequential(); break;
    case 3: searchSequential(); break;
    case 4: createIndexed(); break;
    case 5: searchIndexed(); break;
    case 0: printf("Exiting...\n"); break;
    default: printf("Invalid choice.\n");
}
} while(choice != 0);
return 0;
}

```

Output: -

```

root@fedora: /home/lab1137-33/Desktop# vim file_org.c
root@fedora: /home/lab1137-33/Desktop# gcc file_org.c -o file_org
root@fedora: /home/lab1137-33/Desktop# ./file_org

```

```

--- File Organization Techniques ---
1. Sequential File (Create)
2. Sequential File (Display)
3. Sequential File (Search)
4. Indexed File (Create)
5. Indexed File (Search)
0. Exit
Enter your choice: 1
Enter number of records: 3
Enter ID for record 1: 101
Enter Name for record 1: Alice
Enter ID for record 2: 102
Enter Name for record 2: Bob
Enter ID for record 3: 103
Enter Name for record 3: Charlie
Sequential file created.

```

```

--- File Organization Techniques ---
1. Sequential File (Create)
2. Sequential File (Display)
3. Sequential File (Search)
4. Indexed File (Create)
5. Indexed File (Search)
0. Exit
Enter your choice: 2

```

```

--- Records in Sequential File ---
ID: 101, Name: Alice
ID: 102, Name: Bob
ID: 103, Name: Charlie

```

```

--- File Organization Techniques ---
1. Sequential File (Create)
2. Sequential File (Display)
3. Sequential File (Search)
4. Indexed File (Create)
5. Indexed File (Search)
0. Exit
Enter your choice: 3
Enter ID to search: 102
Record Found: ID: 102. Name: Bob

```

```

--- File Organization Techniques ---
1. Sequential File (Create)
2. Sequential File (Display)
3. Sequential File (Search)
4. Indexed File (Create)
5. Indexed File (Search)
0. Exit
Enter your choice: 4
Enter number of records: 3
Enter ID for record 1: 201
Enter Name for record 1: Zara
Enter ID for record 2: 202
Enter Name for record 2: Yuki
Enter ID for record 3: 203
Enter Name for record 3: Xander
Indexed files created.

```

```

--- File Organization Techniques ---
1. Sequential File (Create)
2. Sequential File (Display)
3. Sequential File (Search)
4. Indexed File (Create)
5. Indexed File (Search)
0. Exit
Enter your choice: 5
Enter ID to search: 202
Record Found: ID: 202, Name: Yuki

```

```

--- File Organization Techniques ---
1. Sequential File (Create)
2. Sequential File (Display)
3. Sequential File (Search)
4. Indexed File (Create)
5. Indexed File (Search)
0. Exit
Enter your choice: 0
Exiting...

```